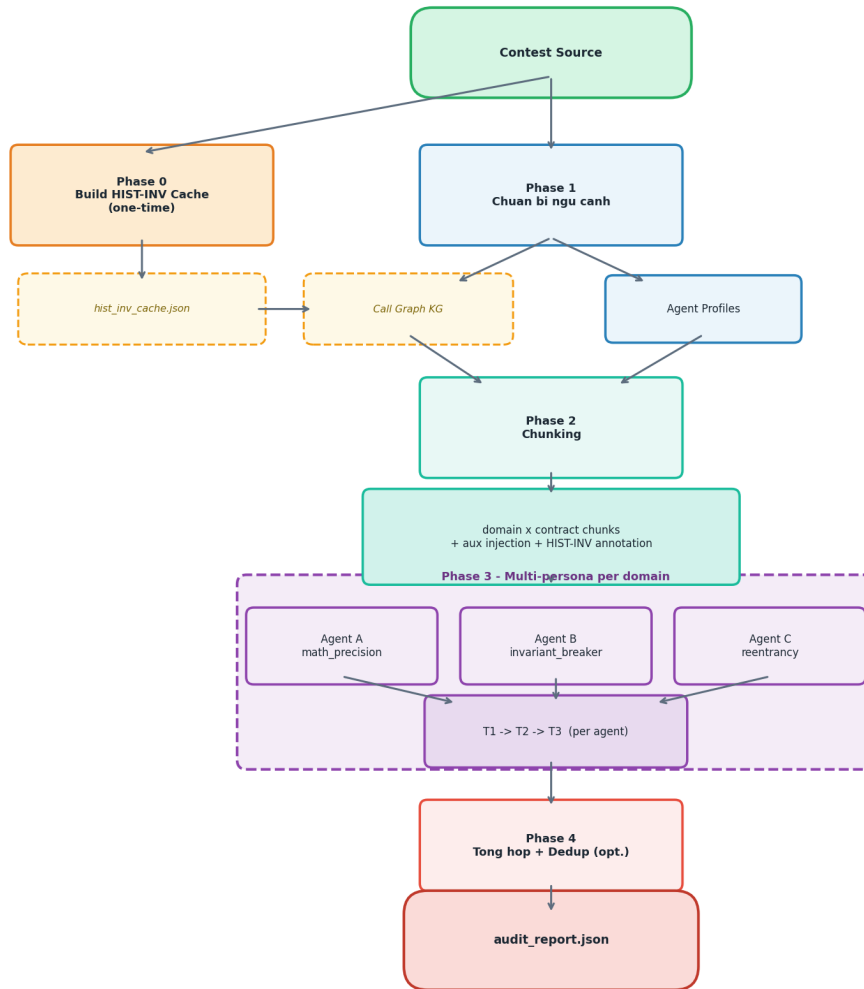


Multi-Expert Contract Audit Panel

I. Kiến trúc Pipeline



Hình 1. Tổng quan kiến trúc pipeline `sim_e2e`

Phase 0 — Build HIST-INV Cache (một lần, dùng lại)

Mục tiêu: Pre-compute invariants cho từng function trong contest, lưu cache để các lần chạy `sim_e2e` sau không tốn thêm LLM call. Script riêng: `populate_hist_inv_cache.py` — chạy một lần trước khi chạy `sim_e2e` lần đầu.

Dữ liệu nguồn: ~3,366 audit findings thực từ Solodit (Code4rena, Sherlock...), mỗi finding được tách thành 4 sections: mô tả lỗi, code snippet, mô tả operation, và invariant statement.

Kiến trúc RAG: 4 ChromaDB collections tương ứng 4 sections. Phase 0 dùng collection `solodit_op` — mô tả thao tác cơ học của từng finding — phù hợp để match với function theo hành vi, không phụ thuộc naming convention của từng contest.

Tác dụng: Agents nhận gợi ý "function này về lịch sử hay bị lỗi kiểu gì" dưới dạng comment [HIST-INV] ngay trong source code, giúp định hướng T2 thay vì scan mù.

Selective invalidation:

- Đổi source một function → xóa entry của function đó, rebuild lại mình nó
- Đổi cả contract → xóa entries của contract đó

- Đổi RAG DB hoặc query logic → xóa toàn bộ và rebuild
- Đổi agent prompt hay config sim_e2e → không cần làm gì

Phase 1 — Chuẩn bị ngữ cảnh

Mục tiêu: Thu thập toàn bộ thông tin về codebase trước khi chạy agents.

- **Đọc contracts** — Scan toàn bộ thư mục, load source code của mọi .sol file.
- **Build call graph (KG)** — Tạo call graph, biết function nào gọi function nào. Kết quả cache lại.
- **Load HIST-INV** — Đọc cache các invariants lịch sử đã match với contest này.
- **Tạo agent profiles** — Sinh danh sách expert agents với persona và system prompt riêng.

Phase 2 — Chia nhỏ bài toán (Chunking)

Mục tiêu: Biến N contracts × M functions thành các bài toán nhỏ hơn để agent có thể focus.

- **Phân loại functions theo domain** — Mỗi function được gán 1 domain. Cùng domain × cùng contract = 1 chunk.
- **Build source cho từng chunk** — Chỉ giữ header + functions cần audit, đính kèm aux + parent contracts.
- **Inject context vào source** — Bổ sung call graph block và [HIST-INV] comments vào từng function.

Phase 3 — Chạy agents song song

Multi-agent per domain: Mỗi domain có nhiều agents với persona khác nhau — ví dụ domain math_cast có math_precision, invariant_breaker, reentrancy_specialist, evm_hardener... Tất cả cùng nhìn vào một đoạn code từ góc độ chuyên môn và chiều sâu khác nhau.

- **T1 — Trích xuất invariants** — Agent tự phát biểu các bất biến cần đúng (INV-1, INV-2...).
- **T2 — Tìm lỗi có hướng dẫn** — Agent dùng invariants T1 + HIST-INV annotations để tìm violations.
- **T3 — Quét độc lập (CoT sweep)** — Agent trace độc lập từng function; chỉ VERDICT=BUG mới report.

Tất cả agent tasks chạy song song qua ThreadPoolExecutor, round-robin qua LLM client pool.

Phase 4 — Tổng hợp kết quả

- **Gom findings per chunk** — T2 + T3 findings của tất cả agents merge lại sau khi agent cuối xong.
- **Dedup (tuỳ chọn)** — Nếu bật --dedup: loại bỏ trùng lặp qua 3 lớp (rule-based → static anchor → LLM judge).
- **Xuất report** — Toàn bộ raw findings → audit_report_{contest_id}_raw.json.

II. Tập dữ liệu test

Nguồn: Web3Bugs — tập hợp các contest audit thực tế từ Code4rena, bao gồm report đầy đủ và source code đã được verify. Tập trung vào **H bugs** (High severity) vì đây là lớp lỗi có tác động tài chính thực sự và được các auditor xác nhận rõ ràng.

Cách chọn contest: Ưu tiên contests có nhiều GT contracts và nhiều H bugs. Contests nhỏ dễ bị ảnh hưởng bởi variance cao.

Đánh giá performance:

- **Thời gian chạy** — Đo wall time theo từng contract (4 workers song song).
- **Recall, Precision, F1** — Tính theo **tổng GT của toàn bộ contracts trong contest**. Lý do: một số contracts chỉ có 1-2 H bugs — nếu miss thì Recall = 0, kéo kết quả tổng xuống không phản ánh đúng năng lực pipeline. Tính gộp làm mịn variance này.

III. Đánh giá thực tế

4 contests benchmark:

Contest	Tên	GT contracts	H bugs
5	Vader Protocol	8	24
35	Trident (Sushi)	4	17
42	Mochi	7	13
104	Joyn	4	9

Kết quả lần chạy gần nhất (raw, no dedup, 4 workers):

Contest	TP	FN	Recall	Precision	F1	Tổng t/g	TB/contract
5	17/24	7	0.708	0.020	0.039	1494s	~187s
35	15/17	2	0.882	0.048	0.090	384s	~96s
42	11/13	2	0.846	0.031	0.059	1002s	~143s
104	8/9	1	0.889	0.036	0.069	644s	~161s

Precision thấp là bình thường: GT chỉ gồm H bugs, nhưng thực tế một contest còn có rất nhiều Medium/Low bugs. Pipeline có thể đang tìm ra các bugs đó nhưng chúng không có trong GT nên bị tính là FP. H bugs được chọn làm GT vì nghiêm trọng nhất, mô tả rõ ràng nhất, và dễ đánh giá match/no-match nhất.

IV. Điểm mạnh và điểm yếu của kiến trúc multi-persona

Điểm mạnh

- **Coverage bù trừ:** Mỗi persona có blind spot riêng. Khi nhiều personas cùng nhìn vào một đoạn code, xác suất có ít nhất một agent bắt được bug tăng đáng kể.
- **Độc lập và không bias lẫn nhau:** Agents không chia sẻ context, không biết agent khác tìm được gì — tránh hiệu ứng anchoring.
- **Chiều sâu chuyên biệt:** Mỗi agent được prompt với kiến thức chuyên sâu về một loại bug cụ thể thay vì một generalist phải cover tất cả.
- **Dễ mở rộng:** Thêm persona mới vào domain không phá vỡ gì, chỉ tăng coverage.

Điểm yếu

- **Sinh ra quá nhiều findings:** Với 5-6 agents × T2 + T3, một chunk có thể sinh 40-100 raw findings, phần lớn trùng lặp hoặc false positive.
- **Chi phí dedup tăng theo số agent:** Bộ dedup 3 lớp tốn thêm thời gian và LLM calls tỉ lệ với số findings đầu vào.

- **Noise che khuất signal:** Report hàng trăm findings khiến người dùng khó phân biệt real bugs với nhiễu.
 - **LLM cost tăng tuyến tính:** Mỗi agent thêm = thêm 3 LLM calls ($T1 + T2 + T3$).
-

V. Các triển khai tiếp theo

Intra-domain verify (vote tính điểm)

Sau khi một chunk hoàn thành, các findings từ tất cả agents trong domain được đưa ra để các agents còn lại trong cùng domain vote — mỗi finding nhận điểm dựa trên số agent xác nhận. Findings điểm thấp bị lọc hoặc hạ mức độ ưu tiên.

Tại sao chỉ verify trong domain, không cross-domain: Agents từ domain khác không có đủ kiến thức chuyên biệt về loại bug đó — chỉ có thể vote mù theo heuristic chung, dẫn đến tăng FN (loại bỏ nhầm findings đúng).

Lợi ích kỳ vọng: Giảm FP đáng kể mà không tăng FN. Findings chỉ xuất hiện ở 1 agent trong khi các agent còn lại không thấy → nhiều khả năng là hallucination. Đây cũng là cơ chế trực tiếp khắc phục điểm yếu cốt lõi của kiến trúc multi-persona: nhiều persona sinh nhiều findings, nhưng chính các persona đó là bộ lọc tốt nhất cho nhau — chỉ findings được nhiều experts đồng thuận mới giữ lại.

Thời điểm chạy: Ngay sau khi chunk xong (trước dedup toàn cục), để vote result có thể feed vào dedup pipeline như tín hiệu bổ sung.